Executive Summary

The design-of-experiments (DOE) software market spans mature commercial tools and growing open-source libraries. Commercial products like **JMP** (SAS), **Minitab**, **Design-Expert** (Stat-Ease) and **Statgraphics** dominate industry and academic use, offering point-and-click interfaces for classical and modern DOE methods. For example, JMP's *Custom Designer* and *Definitive Screening* platforms support full/fractional factorials, Plackett–Burman, response-surface and mixture designs. Minitab and Stat-Ease focus on Six Sigma and R&D workflows; the recently released **Stat-Ease 360** adds space-filling and Gaussian-process (GP) modeling for computer experiments, Python scripting, logistic classification, and split-split-plot support. Tools like **DOE++** (ReliaSoft) cater to reliability engineering with special support for censored lifetime data. Excel add-ins (**DOE Pro XL**, **QI Macros**) provide lightweight DOE inside spreadsheets.

Open-source solutions are centered in R and Python. R's ecosystem (packages such as DoE.base/FrF2, **AlgDesign**, **OptimalDesign**, **rsm**, **DiceDesign**, **lhs**, etc.) covers factorial, mixture, optimal and space-filling designs. Python libraries (e.g. **pyDOE2**, **doepy**) can generate factorial, Plackett–Burman, Box–Behnken, central-composite and various space-filling designs. MATLAB's Statistics Toolbox recently introduced DOE routines (full/fractional, mixture, D-optimal, Taguchi, Latin hypercube, etc.). In summary, the DOE market offers end-to-end GUI tools (with interactive graphs, wizards and integration features) and scripting libraries. Supported designs include 2-level and multi-level factorials (full/fractional), Plackett–Burman, Taguchi arrays, response-surface (CCD, Box–Behnken), mixture experiments, split/split-split plots, robust optimal designs, Latin-hypercubes and custom (D-/I-optimal) designs. Key algorithms include randomization and blocking for run order, alias and confounding analysis, linear/mixed-model regression and ANOVA for analysis, stepwise and exhaustive model selection, D-/I-optimal exchange algorithms, Latin-hypercube space-filling, and GP models for computer experiments.

Commercial DOE packages run on Windows (most) and some support macOS (e.g. JMP) or offer cloud SaaS (Stat-Ease 360). Licensing is typically proprietary (desktop perpetual or subscription), with some academic pricing. Many integrate with other tools: Minitab and JMP can exchange data via CSV/Excel, and both now support calling Python or R code. Stat-Ease 360 and Minitab DOE (Effex) offer cloud-accessible design libraries. Excel-addins leverage the familiar Excel UI. Typical UI/UX features include design wizards (guided experiment setup), interactive factor-level editors, instant graphical diagnostics (factor profiles, contour/surface plots, cube plots) and optimization dialogs. Strengths of leading tools include comprehensive design catalogs and easy visualization; weaknesses may include high cost, Windows-only desktop limitations, or learning curves for advanced features. Open-source toolchains lack polished GUIs but offer scriptability and integration with data pipelines.

Major DOE Software Products (Comparison)

The table below compares key DOE products by supported designs, core features, platforms, integration and UI, plus strengths/weaknesses. Sources are official docs and vendor sites.

Product (Vendor)	Supported Designs / Use-Cases	Key Methods / Analysis	Platforms & Integration	UI / Strengths	Weaknesses	Sources
JMP (SAS)	Full & fractional factorials (2+/n-level), Plackett–Burman, Definitive Screening, Custom (D-/I-optimal), CCD, Box–Behnken, mixture designs, split-plot.	Linear/mixed-model regression, ANOVA, robust regression; interactive model selection (stepwise, all subsets); GP modeling (Pro version).	Windows/macOS desktop; integrates via JSL scripting. Supports ODBC, Excel, Python & R calls.	Highly interactive GUI with *Custom Designer*, dynamic plots and dashboards; strong data visualization; scriptable (JSL).	Expensive; high learning curve for scripts; fewer cloud/collab features.	JMP site (DOE features), user guides.
Minitab (Minitab, Inc.)	Full/fractional factorial, Plackett–Burman, Definitive Screening, split-plot, CCD, Box–Behnken, simplex-lattice/centroid (mixture), Taguchi (L2–L5 mixed), response-surface.	ANOVA/regression, robust regression, GLMs; model selection tools; multivariate/multi-response optimization.	Windows (Desktop); *Minitab Express* on Mac; cloud platform (Minitab Connect). Python integration via `mtbpy`; imports CSV/Excel.	Guided DOE wizards; Minitab Reporting (projects). Extensive support and training.	Primarily GUI (some Python API); moderate cost.	Minitab Help (DOE designs); Minitab DOE by Effex site (catalog, OMARS).
Design-Expert (Stat-Ease)	2-/multi-level factorial, fractional, Plackett–Burman, CCD, Box–Behnken, mixture (simplex lattice/centroid), combined mixture-process, split-plot.	Regression/ANOVA, Poisson/logistic models; ANOVA diagnostics (half-normal, residuals); optimizer with CIs; foldovers and design augmentation.	Windows desktop. XML/Excel import-export. Works with Weibull++ for reliability (via scripting).	Streamlined DOE workflow; rich graphics (3D surfaces, profilers). Good for multi-stage DOE. Lots of custom-design flexibility.	Windows-only; no built-in Python (except in 360); visualization less interactive than JMP.	Stat-Ease (Design-Expert) site; Feature sheet.
Stat-Ease 360 (Stat-Ease)	All Design-Expert designs **plus**: Space-filling (Latin hypercube, maximin), D-/I-optimal, split-split-plot, random-block analysis.	Gaussian Process (Kriging) models for deterministic computer experiments; logistic classification (ROC); Python-based Weibull analysis.	Web/cloud (browser UI). Full Python API (call Stat-Ease engine in Jupyter). Subscription model.	Cutting-edge DOE (cloud) with Python scripting; continuous updates; split-split-plot support.	Requires Internet/subscription; newer product (features evolving).	Stat-Ease 360 product page; 360 feature sheet.
Statgraphics	Full & fractional factorials, Plackett–	Standard regression/ANO	Windows desktop	Wizard-driven DOE	Windows-only;	Statgraphics

Product (Vendor)	Supported Designs / Use-Cases	Key Methods / Analysis	Platforms & Integration	UI / Strengths	Weaknesses	Sources
cs Centurion (Statgraphics Tech.)	Burman, random block designs (RCBD, RCB split-plot, Latin square), definitive screening; CCD, Box–Behnken, Draper–Lin; simplex and extreme-vertex mixture; robust parameter (Taguchi), D-optimal/alias-optimal.	VA; advanced diagnostics; definitive screening analysis; multiresponse optimization.	(v18/19+); .NET architecture. Exports graphs to Office.	with many options; good value.	smaller user base.	website (DOE capabilities).
DOE++ (ReliaSoft)	Factorial, fractional, Taguchi robust (inner/outer arrays), RSM (CCD, Box–Behnken), *Reliability DOE* (censored life data).	ANOVA/regression; specialized analysis for right-censored data; ability to treat failures as responses.	Windows (as part of ReliaSoft Synthesis platform). Can exchange data with Weibull++ for life data analysis.	Handles DOE with censored data – unique in reliability engineering. Integration with FMEA, Reliability tools.	Niche use; modest UI; reliant on ReliaSoft suite; fewer general-purpose features.	StatXpert/ReliaSoft page.
DOE Pro XL (SigmaZone)	2-level full/fractional, 3-level full, Taguchi (L- series), Plackett–Burman, CCD, Box–Behnken, Custom (user-defined).	Multiple linear regression, ANOVA; multiresponse optimization engine.	Microsoft Excel add-in (Windows). Design and results live in Excel sheets; no external file transfers.	Seamless Excel integration – build/modify designs in worksheet; built-in optimizer.	Limited to Excel; UX constrained by spreadsheet; visualization basic.	SigmaZone (DOE Pro) product page.
QI Macros (KnowWare)	Basic factorial (2^k, 3^k), Taguchi (L2–L8), Plackett–Burman screening. (Part of broader Six Sigma Excel toolset.)	Regression/ANOVA; automated control charts & Pareto (SPC-focused).	Excel add-in (Windows; Mac Excel beta). Data in worksheets.	Very easy for Green/Black Belts; “fill-in-the-blank” wizards in Excel; low cost.	Very limited designs; more SPC focus than DOE; basic stats.	QI Macros website (features).
R (CRAN packages)	Full/fractional factorial (DoE.base, FrF2), Plackett–Burman, Taguchi (snr, oa.design), CCD/Box–Behnken (rsm), mixture (AlgDesign, DoE.base), Latin hypercube (lhs, DiceDesign), space-filling (MaxPro, SLHD), split-plot (skpr).	`lm()` / `aov()`, regression and mixed models (lme4); advanced optimal-design (AlgDesign, OptimalDesign); GP (DiceKriging).	Cross-platform (Win/Mac/Linux). Easily calls C/C++ for speed. Interfaces with Rmd, Shiny, Python (via reticulate).	Ultimate flexibility and power; huge community. Many specialized packages (e.g. **skpr** for split-plot optimal designs).	Steep learning curve; no unified GUI (GUI front-ends like Rcmdr exist). Depends on multiple packages.	CRAN Task View – *Experimental Design*.

Product (Vendor)	Supported Designs / Use-Cases	Key Methods / Analysis	Platforms & Integration	UI / Strengths	Weaknesses	Sources
Python (libraries)	Factorial, fractional, Plackett–Burman, Box–Behnken, CCD (pyDOE2); full/fractional factorial, Plackett–Burman, central composite, Box–Behnken, multiple LHS/space-filling options (doepy).	NumPy/SciPy for regression/ANOVA; statsmodels for GLM; SALib for screening analysis; scikit-learn or GPyTorch for GP surrogate models; SciPy optimizers for coordinate exchange.	Cross-platform. Integrates with Jupyter, pandas, Excel (via xlwings). Many pure-Python algos.	Can be embedded in data pipelines and custom apps. OpenTURNS (Python/C++) adds DOE for stochastic simulation.	No native GUI; must script or build custom UI. Requires combining multiple libs.	pyDOE2 PyPI page; doepy docs; SALib, statsmodels docs.
MATLAB (Stats Toolbox)	Full factorial, 2-/3-level fractional, CCD, Box–Behnken, mixture, Taguchi, D-optimal (via `optimalDOE`), Latin hypercube, low-discrepancy (Halton/Sobol).	Regression (`fitlm`), ANOVA, DOE objects (fullFactorialDOE, fractionalFactorialDOE, etc.).	Cross-platform (Windows/Mac/Linux). Tight integration with Simulink. Data Exchange via MAT-files, Excel.	Integrated with Simulink for DOE of simulations. Familiar for MATLAB users.	Proprietary; requires MATLAB + Toolbox. User interface is code-based (some apps exist).	MathWorks documentation.

Common Architectures, Tech Stacks and Reusable Components

Most DOE tools follow a layered architecture with a **frontend** (UI/wizards/plots), a **computation engine**, and data/storage/back-end layers. High-performance languages (C/C++, .NET, or optimized Python/R) are used for heavy tasks (design generation, matrix algebra), while UI layers use GUI frameworks. Modern trends favor web/cloud architectures with RESTful APIs.

Statistical Engine: Open-source projects can leverage libraries like **SciPy/statsmodels** (regression, ANOVA), **GPyTorch** or **scikit-learn** (Gaussian process modeling), and optimization libraries (coordinate-exchange or genetic algorithms for D-/I-optimal design). R libraries (AlgDesign's Federov exchange, OptimalDesign, MixExp) provide mature DOE algorithms. For Latin-hypercube and space-filling designs, libraries such as **lhs** (R) and **doepy**, **pyDOE2** (Python) handle sampling and diversity measures. Parallelization (via multiprocessing or cluster computing) can speed up search.

Data/Database: Relational (PostgreSQL/MySQL) or NoSQL (MongoDB) stores can manage experiment definitions, results, and user projects. Use ORMs or lightweight formats (JSON, CSV) for import/export.

Frontend/UI: Modern web front-ends (React/Angular/Vue) or desktop (Qt, .NET WPF, Electron) frameworks can offer rich interactive graphics (Plotly.js, D3.js) and drag-and-drop experiment builders. Charts (factorial cubes, contour plots) often reuse plotting libraries. For rapid prototyping, Jupyter notebooks or R Shiny can build DOE apps with minimal UI coding.

Interoperability/Integration: Provide APIs or connectors. Notably, Minitab's `mtbpy` and JMP's JSL/Python interfaces allow scripted control. Excel add-ins use COM or XLL plugins. CAD/MES integration (digital twins) could use open standards (CSV, JSON or APIs) or OPC-UA. Analytics pipelines can embed DOE via Python or R modules. For example, a Python backend might use `doepy` to generate designs and return results as pandas DataFrames; calling tools like OpenTURNS is also possible for specialized simulation.

Recommended Libraries/Components: Key open-source libraries include **NumPy/Pandas/SciPy** for core math, **statsmodels** for regression, **pyDOE2/doepy** for design generation, and **SALib** for screening/ANOVA. For R-based implementations, reuse **DoE.base/FrF2** (factorial designs), **AlgDesign/OptimalDesign** (optimal designs), **DiceDesign/lhs** (computer experiment designs) and **rsm** (response surface analysis). On the commercial side, engines from existing software (e.g. calling JMP via JSL or Minitab via Python) could be leveraged if licensing permits.

Proposed DOE Software: Functional Design

To combine best features, a new DOE platform should provide:

- **Core DOE types:** Support 2-/multi-level factorial (full, fractional), Plackett–Burman, screening (including definitive screening), central composite and Box–Behnken RSM, simplex/centroid mixture, Taguchi (L2–L8), and combined mixture-process designs. Include split-plot and split–split-plot for hard-to-change factors. Allow *Custom/Optimal* designs (user-defined candidate sets, D-/I-optimality).
- **Analysis & Modeling:** Built-in regression/ANOVA (including fixed/random effects), GLMs (Poisson, logistic), Box–Cox transformations. Provide model diagnostics (half-normal plots, residual analysis) and model selection (stepwise AIC/BIC, all-subsets). For computer experiments, include surrogate modeling (Gaussian Processes/Kriging) and space-filling design support (Latin hypercube, Sobol/Halton sequences).
- **Randomization & Blocking:** Automate run randomization (with seed control) and allow specification of blocks or split-plots. Compute alias/confounding tables for designs.
- **Optimization & Predictions:** Multi-objective optimization (Desirability, numeric) and simple target search (desirability, ramp functions). Instantaneous prediction plots (factor traces). Leverage algorithms (coordinate exchange, Fedorov's method) for D-/I-optimal design construction.
- **Visualization & UI:** Interactive 2D/3D plots: main-effects, interaction profiles, cube plots for factorial, contour/surface for RSM, simplex graphs for mixtures, profile for LHS. Support drag-and-drop design setup: define factors, levels, and constraints in a form or visual editor. Provide graphical analysis "Dashboard" linking plots and data. Wizards to guide novices (e.g. design recommendations based on goal).
- **Integration & Data I/O:** Import data from CSV/Excel, database or APIs. Export designs/models/results to common formats. Provide APIs/SDK for Python, R, and COM (Excel) – for example, a RESTful backend or embedded scripting (like JMP's JSL). Connectors for MES/PLM systems (e.g. via REST APIs or OPC) and CAD (e.g. parameter import).
- **Reproducibility & Audit:** Log all steps and random seeds. Save entire workflows (design specs, data, analysis) in project files (as Minitab's .mpj or JSON). Enable versioning of experiments and

keep an audit trail of who did what and when.

- **Collaboration:** Multi-user support with roles. Share experiments and results in real time (web sockets or messaging). Commenting on designs. Optionally, a web dashboard for teams.
- **Scalability & Performance:** Architect for horizontal scalability (microservices). Offload heavy design searches to parallel threads or cloud workers. Use optimized math libraries (BLAS/MKL). Consider GPU acceleration for large optimization (optional).

Architecture: A recommended modular stack is:

```
mermaid


graph LR
    subgraph Frontend
        UI[Browser/Desktop UI]
    end
    subgraph Backend
        API[Application Server/API]
        DB[(Database)]
        Engine[DOE Engine (Python/R)]
        Integrations[Integration Layer]
    end
    UI --> API
    API --> Engine
    API --> DB
    API --> Integrations
    Engine --> DB
    Integrations --> {CAD/ MES}
    Engine --> {Python Scripts / R Scripts}
```

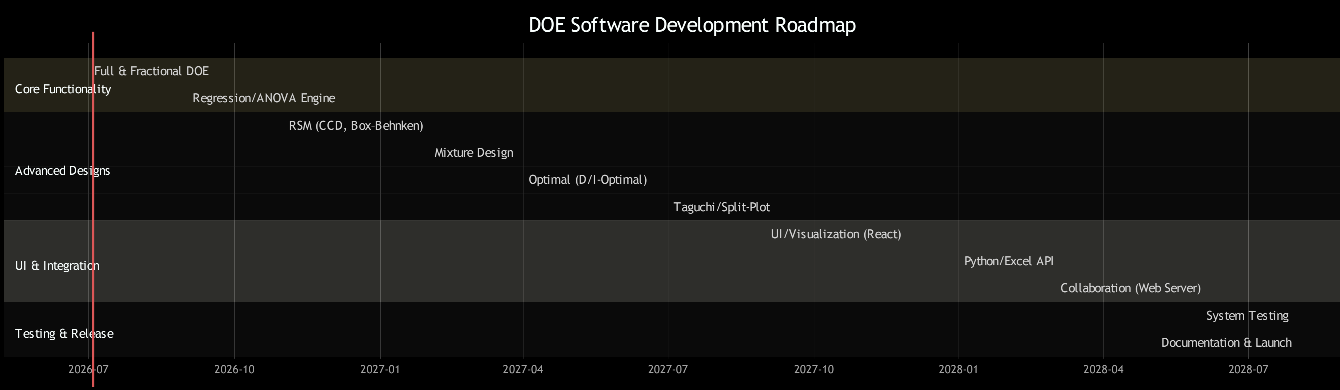
- **Frontend:** e.g. a React or Electron-based GUI for drag-drop factor setup, wizards, and plotting.
- **Backend/API:** Python (Flask/FastAPI) or R (Plumber/Shiny) providing REST endpoints. Engine can call R libraries (via rpy2 or Rserve) or pure-Python modules.
- **Database:** Store experiment definitions and results (PostgreSQL/Mongo).
- **Engine:** Statistical computations using NumPy/SciPy, statsmodels, or R's lm/aov. Implement design algorithms (coordinate exchange, genetic algorithms) via open libraries or custom code.
- **Integrations:** Services to import/export Excel, call CAD/MES APIs, and execute user-defined scripts (Jupyter integration).

Development Roadmap and Milestones

A phased development plan (assuming a small team) could be:

- **M1 (Core DOE, ~4 PMs):** Implement basic factorial/full-factorial design generation, ANOVA/regression analysis, simple plots. Integrate CSV/Excel import/export and a minimal UI for factor entry.

- **M2 (RSM & Mixture, ~5 PMs):** Add central composite and Box–Behnken design generation, polynomial model fitting, RSM contour/surface plots. Support simplex-lattice/centroid mixture designs and pseudo-components.
- **M3 (Optimal Designs, ~4 PMs):** Implement D- and I-optimal design routines (using coordinate-exchange); add Latin hypercube and space-filling design generation. Provide optimization interface.
- **M4 (Advanced Models, ~3 PMs):** Include split/split-plot design handling, random-block analysis, Taguchi arrays, GLMs (logistic). Integrate Gaussian-process modeling for computer experiments.
- **M5 (UI/UX & Integration, ~6 PMs):** Build full-featured GUI (wizards, drag-drop). Add interactive graphics (Plotly). Develop Python/R API and an Excel add-in connector. Implement user/project management features (saving, audit logs).
- **M6 (Testing & Refinement, ~3 PMs):** Intensive unit/integration testing, benchmark accuracy against reference (Minitab/JMP). Improve documentation and user help.
- **Total ~25 person-months** (rough estimate, assuming developers can multitask).



Testing and Validation: Each module will have unit tests (e.g. check generated designs match known orthogonal arrays) and regression tests (compare analysis results with Minitab/JMP). System testing includes end-to-end experiments from design through analysis. We recommend cross-validating models and using synthetic data to verify model-selection and ANOVA routines. Beta testing with typical workflows (e.g. industrial case studies) will ensure reliability and usability.

Feature Matrix and MVP Scope

Key features are prioritized by value and effort:

Feature	Priority	Notes
2-level full & fractional factorials	High	Foundation for DOE; must-have.
Plackett–Burman screening	High	Common screening design.
CCD & Box–Behnken (RSM)	High	Core optimization designs.
Mixture designs (simplex)	Medium	Important in formulations; after core.

Feature	Priority	Notes
Regression/ANOVA (LM) & diagnostics	High	Basic analysis tools.
Taguchi (L-arrays)	Low/Med	Specialized; can follow essentials.
D- and I-optimal design	Medium	Powerful; adds completeness.
Latin Hypercube/Space-filling	Medium	For simulations; advanced use-case.
Split-plot & blocking	Medium	Real-world applicability.
Interactive plots (main effects, contours)	High	Enhances understanding (base feature).
DOE wizards (guided UI)	Medium	Improves usability.
Multiresponse optimization (desirability)	Medium	Adds completeness for multiple goals.
Python/R integration API	Medium	Important for extensibility.
Collaboration (projects, sharing)	Low	Useful, but can be deferred post-MVP.

MVP Scope: Implement the **high-priority features**. An MVP should allow users to define factors/levels, create 2-level (and fractional) designs, run experiments (input data), and perform regression/ANOVA analysis with standard DOE plots and one-at-a-time **prediction profiles**. CSV/Excel import-export and basic project save/load ensure reproducibility. In summary, core DOE generation, analysis, and reporting form the MVP. Advanced features (optimal designs, space-filling, collaboration) can be added in later releases once the core is stable.

Throughout development, all user actions should be logged (for audit trails) and results reproducible by storing random seeds and versioned data. The system should be thoroughly documented, with example notebooks or tutorials illustrating typical DOE workflows.

Diagram: System Architecture (Mermaid)

```
mermaid

graph LR
    UI[User Interface (Web/App)] -->|sends requests| API[Backend API]
    API -->|requests analysis| Engine[Statistical Engine (Python/R)]
    API -->|reads/writes| DB[(Database)]
    API -->|invokes| Integrations[Integration Layer]
    Engine --> DB
    Integrations -->|imports/exports| Excel/CSV/SQL
    Integrations -->|calls| PythonScripts
```

This architecture emphasizes modularity: a clean separation between UI, server logic, and computation. The integration layer handles data I/O and external tools.

Timeline Chart (Mermaid) (as shown above under Roadmap).

References

Key information was drawn from official sources: JMP and Stat-Ease product pages, Minitab support/docs, SigmaZone (DOE Pro), ReliaSoft (DOE++), Statgraphics website, and CRAN/PyPI documentation for R/Python libraries. These provided the features, supported designs, algorithms, and integration capabilities summarized above.